

ParkHub

Smart City Geolocation Parking Discovery & Integrated Digital Wallet
Platform



A Comprehensive Architectural Specification, Technical Reference, Database Schema
Analysis, and Local XAMPP Deployment Guide.

Prepared by: Antigravity AI Engineering & Senior Software Architect Group

Version: 1.0.0 (Production Grade)

Platform Stack: Laravel 11 (PHP 8.2+) / React 18 + Astro 6 (JavaScript) / MySQL (XAMPP)

Date: May 2026

1. Project Overview & Architectural Paradigm

ParkHub (localized for regional operations as **ParkIndia**) is a high-availability, real-time B2C parking reservation, discovery, and smart-city management platform. It addresses critical urban logistics challenges: traffic congestion caused by vehicles searching for parking spaces, fuel wastage, billing complexity, dynamic demand management, and lack of visual booking space control.

1.1 Core Problems Solved

- **Urban Traffic Mitigation:** Reduces block-circling behavior by showcasing real-time occupancy.
- **Financial Transparency:** Replaces manual paper ticketing with atomic credit wallet accounting.
- **User-Experience Friction:** Eliminates spot-hunting through precise 2D bay visual selection.
- **System Integration:** Bridges software bookings with physical barrier gates via hardware webhooks.

1.2 Key System Capabilities

- **Interactive GIS Map:** Employs Leaflet.js map view with customized coordinates and color markers to display spot counts across cities.
- **2D Floor Layout Grid:** Generates responsive grids of parking bays categorized by zone parameters (Standard, VIP, EV Charging, Accessible).
- **Atomic Wallet System:** Manages credit balances within a digital transactional ledger, incorporating automatic refunds on user cancellation.
- **Simulated Gate Controller webhooks:** Automated Number Plate Recognition (ANPR) cameras trigger check-in gates instantly on plate match.

1.3 Technology Stack Profile

Layer	Technologies	Architectural Purpose
Frontend Client	React 18, Astro 6, Leaflet.js, Axios	Single Page Application (SPA), GIS map canvas, client-side route flow.
Backend Gateway	Laravel 11, PHP 8.2+, Eloquent ORM	REST API, transactional business rules engine, database gateway.
Authentication	Laravel Sanctum, Bcrypt Hashing	Stateful cookie and secure bearer-token session authorization.
Database & Cache	MySQL 8.0, XAMPP, phpMyAdmin	Relational transaction storage with acid isolation, schema versioning.
Utility Addons	Dompdf 3.1, Chillerlan QR Code	PDF invoice compilation, secure entry-pass QR code rendering.

2. Complete Folder Structure Mapping

The codebase is partitioned into a **Laravel RDBMS API core** (root project) and a modular **Astro/React SPA view client** (contained within `/parkhub-web`). Below is a comprehensive blueprint detailing every directory and critical file:

2.1 Backend Project Directory Structure (Laravel)

```
parkhub-php/
├─ app/
│   ├─ Console/                # Cron scheduling and background Artisan engines
│   │   └─ Commands/
│   │       └─ CheckParkingAvailability.php    # Syncs real-time occupancy counts
│   ├─ Http/
│   │   ├─ Controllers/
│   │   │   └─ Api/            # Unified REST Controllers (69 controllers)
│   │   │       ├─ AuthController.php        # User Register & Login processing
│   │   │       ├─ BookingController.php     # Main booking transaction controller
│   │   │       ├─ AdminAnalyticsController.php # Dashboard metric visual compiler
│   │   │       └─ WalletController.php      # Credit Wallet balance & transaction management
│   │   └─ Middleware/
│   │       └─ EnsureUserIsAdmin.php         # REST administration gatekeeper
│   │   └─ Requests/                # Payload verification schemas (FormRequests)
│   └─ Models/                    # Eloquent Database RDBMS Models
│       ├─ User.php, Booking.php, ParkingLot.php, ParkingSlot.php
│       └─ CreditTransaction.php, Vehicle.php, AuditLog.php, Zone.php
├─ Services/                    # Business Logic & Calculation Engines
│   ├─ BookingCreationService.php        # Atomic booking validation and creation
│   ├─ ParkingAvailabilityService.php     # Real-time space cache resolver
│   └─ PricingEngineService.php          # Dynamic hourly and daily cost engine
├─ bootstrap/                   # Core framework configuration bootstrapper
├─ config/                     # App environment configuration files
├─ database/
│   ├─ migrations/             # RDBMS database schema version migrations
│   └─ seeders/                # Database seed data generators
│       ├─ IndianParkingLotsSeeder.php   # Seeds 15 Indian metropolitan lots
│       └─ ProductionSimulationSeeder.php # Runs large-scale simulation scripts
├─ routes/
│   ├─ api.php                 # REST API Routing paths (Sanctum protected)
│   └─ web.php                 # Standard web views catch-all router
└─ public/                    # Document root folder, exposes compiled web assets
```

2.2 Frontend SPA Directory Structure (Astro + React)

```
parkhub-php/parkhub-web/
├─ public/                # Static assets (images, icons, SVGs, favicon)
├─ src/
│   ├─ App.jsx            # React router shell and layout manager
│   ├─ api/
│   │   └─ client.js      # Axios HTTP client, handles token injection headers
│   ├─ components/        # Reusable shared components (Navbar, Sidebar, Modals)
│   ├─ hooks/             # Custom React Hooks
│   │   ├─ useAuth.js     # Global authentication state hook
│   │   └─ useWallet.js   # Wallet transaction & balance hook
│   ├─ pages/             # Astro routes
│   ├─ views/             # Module-specific view components
│   │   ├─ Login.jsx      # Login & sign-up form screens
│   │   ├─ MapView.jsx    # Leaflet.js geolocation maps & search pins
│   │   ├─ Bookings.jsx   # User booking management and invoice views
│   │   ├─ Wallet.jsx     # Glassmorphic digital wallet & credit top-up panel
│   │   └─ AdminAnalytics.jsx # Admin chart visualization center
│   └─ styles/            # Global variables, HSL color tokens, and custom styles
├─ astro.config.mjs       # Build system configuration
└─ package.json           # NPM package manifests
```

3. The MVC Request Lifecycle Flow

The ParkHub platform uses a decoupled Model-View-Controller (MVC) architecture. The client frontend handles the View layer, transmitting requests via REST APIs to the Laravel Model-Controller backend.

```
[ React View Client ] (axios HTTP Request) || || 1. POST /api/bookings { lot_id, slot_id, times, vehicle_plate } \/ [ REST Route Gateway: routes/api.php ] || || 2. Verifies Session & Sanctum Bearer Token \/ [ Middleware Pipeline: auth:sanctum ] || || 3. Parses Payload into FormRequest Validates \/ [ API Controller: app/Http/Controllers/Api/BookingController@store ] || || 4. Instantiates Business Services Engine \/ [ Business Service: app/Services/BookingCreationService ] || || 5. Executes DB Transaction (FOR UPDATE atomic row locking) \/ [ Database Persistence: MySQL & InnoDB Tables ] ===(Save & Commit)==> [ DB Storage ] || || 6. Serializes Database Records as JSON \/ [ Controller JSON Resource Response ] || || 7. Returns HTTP 200 OK success payload \/ [ React View Client ] (Renders check-in QR pass and updates wallet balance)
```

3.1 Request Lifecycle Stages

- Request Dispatch:** A user selects a slot (e.g. "A-1" at BKC Plaza) from 14:00 to 16:00 for vehicle MH01AB1234. The browser sends a POST request to /api/bookings.
- Routing and Security Middlewares:** routes/api.php routes the request through a Sanctum middleware barrier. If authenticated, the user profile is loaded into context.
- Payload Sanitization:** Input is verified by a request validator (FormRequest), checking date rules and confirming the vehicle plate matches valid state formats.
- Service Execution:** The controller invokes the BookingCreationService. A MySQL transaction begins, locking the target slot record using FOR UPDATE to prevent simultaneous bookings by other threads.
- Atomic Financial Ledgering:** The system calculates cost, verifies that the user's wallet balance covers it, deducts credits, writes an audit record, and updates the slot status to `reserved`.
- JSON View Compilation:** The database changes commit, and the API controller compiles a formatted JSON response containing invoice details and a check-in QR code token.

4. API & Web Routing Engine

Routing endpoints are divided into web and API channels, providing secure access and path separation.

4.1 Route File Definitions

- **Web Router (routes/web.php)**: Used for system health checks, Blade file renders, and client single-page-application (SPA) fallback loaders.
- **API Router (routes/api.php)**: Handles all REST controllers, grouped by authentication requirements.

4.2 Important API Routes Reference Table

Method	URI Endpoint	Target Controller Action	Purpose
POST	/api/login	AuthController@login	Authenticates credentials, returns session token.
POST	/api/register	AuthController@register	Creates user, grants base wallet credits.
GET	/api/availability	ParkingAvailabilityController@index	Lists active slots across lots.
GET	/api/cities	CityController@index	Lists supported Indian metropolitan hubs.
GET	/api/bookings	BookingController@index	Retrieves active and historic bookings.
POST	/api/bookings	BookingController@store	Creates a booking and deducts wallet balance.
DELETE	/api/bookings/{id}	BookingController@destroy	Cancels reservation, processes balance refunds.
POST	/api/bookings/{id}/check-in	BookingCheckInController@checkin	Logs check-in time and sets slot status.
GET	/api/user/wallet/balance	WalletController@getBalance	Returns user's current wallet credits balance.
POST	/api/user/wallet/add	WalletController@addCredits	Top-up user's digital wallet balance.
GET	/api/admin/analytics/overview	AdminAnalyticsController@overview	Compiles admin dashboard analytics metrics.
POST	/api/webhooks/hardware/anpr	HardwareWebhookController@handleAnpr	Simulates boom gate opening on plate match.

[!] Route Rate-Limiter Policy: Public route groups (like login/register and webhook inputs) are protected by a throttle middleware (default: throttle:60,1) to prevent brute-force attacks.

5. Controller Architecture & Core Controllers

The API layer is structured with single-responsibility controllers residing in `app/Http/Controllers/Api/`. Below is a detailed breakdown of the three key controller divisions:

5.1 Authentication Controller (`AuthController.php`)

Manages user registration, secure session creation, user profile updates, and active session verification.

- `login(Request $request)` : Validates credentials, checks accounts, logs users in, updates ``last_login``, and returns a session token.
- `register(Request $request)` : Creates user models, hashes passwords using Bcrypt, sets default wallet balance quotas, and automatically generates session tokens.
- `logout(Request $request)` : Revokes current Sanctum bearer tokens and ends server sessions.
- `me()` : Returns current user statistics, wallet balance, and user roles (user, admin, superadmin).

5.2 Booking Controller (`BookingController.php`)

Coordinates space validation, processes dynamic pricing calculations, handles wallet balance payments, and logs transaction files.

- `index(Request $request)` : Returns paginated lists of user bookings, filterable by date and booking status.
- `store(StoreBookingRequest $request)` : The core booking handler. Runs payload validations, verifies slot availability, processes wallet payment, updates slot state, and creates the booking record.
- `destroy($id)` : Processes cancellations. Releases the reserved slot, calculates the refund amount, and inserts an audit log entry.
- `quickBook(Request $request)` : Instantly finds and allocates the nearest available parking space in a selected lot.

5.3 Administrative Suite (`AdminAnalyticsController.php`)

Provides analytics dashboard data and reports for administrators and lot operations staff.

- `overview()` : Computes multi-lot KPI statistics: Total Revenue, Occupancy Rate, Total Bookings, Active Vehicles.
- `occupancy()` : Yields historical slot occupancy curves to identify peak hour congestion.
- `revenue()` : Summarizes daily, weekly, and monthly payment transactions to track earnings.
- `popularLots()` : Identifies top-performing hubs by reservation count and parking turnover.

6. Database Schema Blueprint (Very Detailed)

The database schema is designed for relational consistency and fast lookups, utilizing InnoDB engines for transaction support.

6.1 RDBMS Table Specifications

Table: users

Column Name	Data Type	Constraints / Keys	Description
id	char(36) (UUID)	Primary Key	Unique user identifier.
name	varchar(255)	Not Null	User's full name.
email	varchar(255)	Unique, Index	Email address used as login ID.
password	varchar(255)	Not Null	Bcrypt-encrypted password hash.
role	varchar(50)	Default: 'user'	Role flag: user , admin , superadmin .
credits_balance	int	Default: 0	Unified digital wallet balance.
credits_monthly_quota	int	Default: 40	Allocated monthly free quota limit.

Table: parking_lots

Column Name	Data Type	Constraints / Keys	Description
id	char(36) (UUID)	Primary Key	Unique lot identifier.
name	varchar(255)	Not Null	Hub name (e.g. BKC Plaza).
city	varchar(100)	Index	City name for filtering.
latitude	decimal(10, 8)	Not Null	Geographic latitude coordinate.
longitude	decimal(11, 8)	Not Null	Geographic longitude coordinate.
total_slots	int	Not Null	Total parking capacity.
available_spots	int	Default: 0	Real-time available parking space count.
available_slots	int	Default: 0	Real-time available parking slot count (legacy/fallback).
hourly_rate_inr	decimal(10, 2)	Default: 0.00	Pricing rate in INR per hour.

Table: parking_slots

Column Name	Data Type	Constraints / Keys	Description
id	char(36) (UUID)	Primary Key	Unique slot identifier.
lot_id	char(36)	Foreign Key, Index	Links to parking_lots.id .
slot_number	varchar(50)	Not Null	Unique code name (e.g., A-1, A-2).
status	varchar(50)	Default: 'available'	Status: available , occupied , reserved .
reserved_for_department	varchar(100)	Nullable	Dedicated parking label (EV, VIP, Accessible).

Table: bookings

Column Name	Data Type	Constraints / Keys	Description
id	char(36) (UUID)	Primary Key	Unique booking identifier.
user_id	char(36)	Foreign Key, Index	Links to users.id .
slot_id	char(36)	Foreign Key, Index	Links to parking_slots.id .
start_time	timestamp	Not Null	Planned check-in start time.
end_time	timestamp	Not Null	Planned check-out end time.
status	varchar(50)	Default: 'confirmed'	Status: confirmed , cancelled , completed .
vehicle_plate	varchar(20)	Not Null	Licensed vehicle number plate.

Table: credit_transactions

Column Name	Data Type	Constraints / Keys	Description
id	char(36) (UUID)	Primary Key	Unique transaction identifier.
user_id	char(36)	Foreign Key, Index	Links to users.id .
amount	int	Not Null	Balance change (positive = top-up, negative = deduction).
type	varchar(50)	Not Null	Transaction type (grant, deduction, refund).
description	varchar(255)	Nullable	Details describing the transaction.

7. End-to-End Data & Transactional Flow

This section traces the transactional steps of two critical system operations.

7.1 Dynamic Booking and Transactional Balance Deduction

When a user books a spot, the system uses a database transaction to lock rows and ensure wallet updates are atomic.

```
DB::transaction(function() use ($request, $user) {
    // 1. Fetch and lock slot row to prevent concurrent bookings
    $slot = ParkingSlot::lockForUpdate()->findOrFail($request->slot_id);

    if ($slot->status !== 'available') {
        throw new SlotOccupiedException("Selected slot is no longer available.");
    }

    // 2. Compute dynamic cost
    $cost = PricingEngine::calculate($slot->lot_id, $request->start, $request->end);

    // 3. Verify user wallet balance
    if ($user->credits_balance < $cost) {
        throw new InsufficientBalanceException("Insufficient credits in wallet.");
    }

    // 4. Deduct balance atomically
    $user->decrement('credits_balance', $cost);

    // 5. Create the booking record
    $booking = Booking::create([
        'user_id' => $user->id,
        'slot_id' => $slot->id,
        'start_time' => $request->start,
        'end_time' => $request->end,
        'vehicle_plate' => $request->plate,
        'status' => 'confirmed'
    ]);

    // 6. Log the financial ledger transaction
    CreditTransaction::create([
        'user_id' => $user->id,
        'booking_id' => $booking->id,
        'amount' => -$cost,
        'type' => 'deduction',
        'description' => "Booking charge for Slot: {$slot->slot_number}"
    ]);

    // 7. Update slot availability status
    $slot->update(['status' => 'reserved']);
});
```

7.2 Automatic ANPR Gate Entry Webhook Integration

The system supports automated gate check-ins. When a camera detects a vehicle's license plate, it calls the hardware webhook:

```
POST /api/webhooks/hardware/anpr
{ "plate": "MH12AB1234", "timestamp": "2026-05-18T10:00:00Z" }
```

The backend processes the webhook as follows:

1. Searches for active, confirmed bookings matching the detected vehicle plate number:
`Booking::where('vehicle_plate', $plate)->where('status', 'confirmed')->first();`
2. If a matching booking is found, the system:
 - Updates the booking check-in time: `$booking->update(['checked_in_at' => now()]);`
 - Updates the physical slot status to occupied: `$booking->slot->update(['status' => 'occupied']);`
 - Sends a push notification or SMS alert confirming check-in.
 - Triggers the barrier gate open command via HTTP response.
3. If no matching booking is found, the gate remains closed and a log is written.

8. System Security, Validation & Auth Core

ParkHub implements multiple security layers to protect the system and ensure reliable data input validation.

8.1 Authentication Framework

- **Laravel Sanctum:** Provides secure session token and stateful cookie management, preventing unauthorized API requests.
- **Bcrypt Hashing:** Passwords are encrypted before database insertion, using a work factor of 12 for strong protection.
- **Middleware Gateways:** Restricts access to administrative endpoints using role-based routing middleware.

8.2 Input Payload Validation Matrix

API Endpoint	Validation Guardrails (FormRequest)	Architectural Purpose
/api/register	'name' => 'required string max:255' 'email' => 'required email unique:users' 'password' => 'required min:8 confirmed'	Validates new user profiles and email uniqueness, preventing duplicate accounts.
/api/bookings	'slot_id' => 'required exists:slots,id' 'start_time' => 'required date after:now' 'vehicle_plate' => 'regex:/^[A-Z]{2}\d{2}[A-Z]{1,2}\d{4}\$/'	Validates the slot ID and ensures the license plate matches standard Indian vehicle formats (e.g. MH12AB1234).
/api/user/wallet/add	'amount' => 'required integer min:100 max:10000'	Enforces minimum (₹100) and maximum (₹10,000) limits for digital wallet top-ups.

9. Visual Interface Showcase (Live System Screenshots)

The following screenshots showcase the live, running web client interface under the user account `dhirendrahudda@gmail.com`:

Map View Screenshot

Figure 9.1: Geolocation interactive map showing custom-colored lot availability pins in metro regions.

Bookings History Screenshot

Figure 9.2: Booking history dashboard containing active, upcoming, and past check-in transactions.

Wallet Dashboard Screenshot

Figure 9.3: Glassmorphic prepaid transaction ledger panel showing top-ups and reservation deductions.

10. Local XAMPP Environment Bootstrapping Guide

To run ParkHub locally in a Windows development environment using XAMPP:

1. Open the **XAMPP Control Panel** and start both **Apache** and **MySQL**.
2. Open ****phpMyAdmin**** in your browser at `http://localhost/phpmyadmin`.
3. Create a new empty database named `parkindia` with the collation set to `utf8mb4_unicode_ci`.
4. Create a `.env` configuration file in the project root directory and update the database settings:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=parkindia
DB_USERNAME=root
DB_PASSWORD=
```

5. Install PHP dependencies and run the database migrations and seeders:

```
composer install
php artisan migrate:fresh --seed --class=IndianParkingLotsSeeder
```

6. Start the Laravel API backend server:

```
php artisan serve
```

7. Navigate to the frontend directory, install npm dependencies, and start the React dev server:

```
cd parkhub-web
npm install
npm run dev
```

8. Open your browser and navigate to `http://localhost:4321` to access the application.

11. Future Scalability & Hardware Integration Road Map

To scale ParkHub for higher transaction volumes and integrate physical hardware, we recommend the following roadmap:

11.1 Database Performance Optimization

- **Read-Write Splitting:** Set up database replicas where write operations go to a master node, and heavy administrative dashboard read queries are distributed to read replica nodes.
- **Database Partitioning:** Partition the `bookings` table by range based on the `start_time` column. This keeps query times fast even as the table grows to millions of records.

11.2 Queue Management

Offload long-running tasks, such as generating invoice PDFs and sending email receipts, from the request-response cycle to background workers using Laravel Queues with Redis:

```
dispatch(new GenerateInvoicePdfJob($booking));
```

11.3 Smart City Integration (Physical ANPR Cameras)

Replace simulated hardware webhook inputs with physical ANPR cameras deployed at parking entry and exit gates. A production-grade ANPR gateway can handle high-performance license plate recognition and control barrier gates in real time:

```
+-----+ +-----+ | Incoming Car | | ANPR Camera Unit | | at Entry Gate | ==(Scans Plate)==>|
(Reads Plate) | +-----+ +-----+ || || (POST API Webhook) \ / +-----+
+-----+ | Physical Gate | | ParkHub Backend | | Barrier Opens |<==(HTTP Response)=| (Matches Booking | |
| Trigger Open | & Wallet Balance) | +-----+ +-----+
```

11.4 High Availability API Servers

Deploy the Laravel API behind a load balancer (such as HAProxy or AWS Application Load Balancer) with Laravel Octane running on Swoole or RoadRunner. This configuration allows the backend to handle thousands of requests per second with low latency.